In [4]:
```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk import pos_tag
from nltk.stem import PorterStemmer, WordNetLemmatizer
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
```

In [13]:
```
import nltk

nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('wordnet')
```

```
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\ADMIN\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\ADMIN\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\ADMIN\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     C:\Users\ADMIN\AppData\Roaming\nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]       date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     C:\Users\ADMIN\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping taggers\averaged_perceptron_tagger_eng.zip.
[nltk_data] Downloading package wordnet to
[nltk_data]     C:\Users\ADMIN\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
```

Out[13]: True

In [14]:
```
document = "Natural Language Processing is a fascinating field. It deals with under
```

In [15]:
```
# 1. Tokenization
tokens = word_tokenize(document)
print("Word Tokens:", tokens)
```

```
Word Tokens: ['Natural', 'Language', 'Processing', 'is', 'a', 'fascinating', 'fiel
d', '.', 'It', 'deals', 'with', 'understanding', 'and', 'generating', 'human', 'lang
uage', 'using', 'computers', '.']
```

In [16]:
```
# 2. POS Tagging
pos_tags = pos_tag(tokens)
print("\nPOS Tags:", pos_tags)
```

```
POS Tags: [('Natural', 'JJ'), ('Language', 'NNP'), ('Processing', 'NNP'), ('is', 'VB
Z'), ('a', 'DT'), ('fascinating', 'JJ'), ('field', 'NN'), ('.', '.'), ('It', 'PRP'),
('deals', 'VBZ'), ('with', 'IN'), ('understanding', 'JJ'), ('and', 'CC'), ('generati
ng', 'VBG'), ('human', 'JJ'), ('language', 'NN'), ('using', 'VBG'), ('computers', 'N
NS'), ('.', '.')]
```

In [17]:
```
# 3. Stop Words Removal
stop_words = set(stopwords.words('english'))
filtered_words = [word.lower() for word in tokens if word.lower() not in stop_words
print("\nAfter Stop Words Removal:", filtered_words)
```

```
After Stop Words Removal: ['natural', 'language', 'processing', 'fascinating', 'fiel
d', 'deals', 'understanding', 'generating', 'human', 'language', 'using', 'computer
s']
```

In [18]:
```
# 4. Stemming
stemmer = PorterStemmer()
stemmed_words = [stemmer.stem(word) for word in filtered_words]
print("\nStemmed Words:", stemmed_words)
```

```
Stemmed Words: ['natur', 'languag', 'process', 'fascin', 'field', 'deal', 'understan
d', 'gener', 'human', 'languag', 'use', 'comput']
```

In [19]:
```
# 5. Lemmatization
lemmatizer = WordNetLemmatizer()
lemmatized_words = [lemmatizer.lemmatize(word) for word in filtered_words]
print("\nLemmatized Words:", lemmatized_words)
```

```
Lemmatized Words: ['natural', 'language', 'processing', 'fascinating', 'field', 'dea
l', 'understanding', 'generating', 'human', 'language', 'using', 'computer']
```

In [20]:
```
# 6. Term Frequency (TF)
cleaned_text = " ".join(lemmatized_words)

count_vectorizer = CountVectorizer()
tf_matrix = count_vectorizer.fit_transform([cleaned_text])

print("\nTerm Frequency Matrix:\n", tf_matrix.toarray())
print("TF Feature Names:", count_vectorizer.get_feature_names_out())
```

```
Term Frequency Matrix:
 [[1 1 1 1 1 1 2 1 1 1 1]]
TF Feature Names: ['computer' 'deal' 'fascinating' 'field' 'generating' 'human' 'lan
guage'
 'natural' 'processing' 'understanding' 'using']
```

In [21]:
```
# 7. TF-IDF
tfidf_vectorizer = TfidfVectorizer()
tfidf_matrix = tfidf_vectorizer.fit_transform([cleaned_text])

print("\nTF-IDF Matrix:\n", tfidf_matrix.toarray())
print("TF-IDF Feature Names:", tfidf_vectorizer.get_feature_names_out())
```

```
TF-IDF Matrix:
 [[0.26726124 0.26726124 0.26726124 0.26726124 0.26726124 0.26726124
  0.53452248 0.26726124 0.26726124 0.26726124 0.26726124]]
TF-IDF Feature Names: ['computer' 'deal' 'fascinating' 'field' 'generating' 'human'
'language'
 'natural' 'processing' 'understanding' 'using']
```